

Deep RNNs | Stacked LSTMs | Stacked GRUs

1. Introduction: What Is a Deep RNN?

So far, we've seen a **single RNN layer**, where each time step's output depends on:

- The **current input** (x_t)
- The **previous hidden state** (h_{t-1})

That's called a **shallow RNN** because there's just one recurrent layer.

But for complex tasks (like speech recognition, translation, or video understanding), one layer may not be enough to capture all the hierarchical features.

So we “stack” multiple RNN layers, **output of one becomes input to the next**.

This gives us a **Deep or Stacked RNN**.

2. Architecture: How Deep RNNs Work

In a **stacked RNN**, you have multiple recurrent layers, each processing the sequence at a higher level of abstraction.

Example (3-layer RNN):

Input sequence: $x_1, x_2, x_3, \dots$

Layer 1 (RNN_1): learns basic temporal patterns

↓

Layer 2 (RNN_2): learns higher-order features

↓

Layer 3 (RNN_3): learns complex dependencies

↓

Output layer (Dense or Softmax)

Each layer passes its **hidden states** to the next layer, not just the final output.

So:

$$h_t^{(1)} = f(W^{(1)}x_t + U^{(1)}h_{t-1}^{(1)})$$

$$h_t^{(2)} = f(W^{(2)}h_t^{(1)} + U^{(2)}h_{t-1}^{(2)})$$

and so on.

Each superscript (¹, ², ³) refers to a layer.

3. Why Stack RNNs?

Because **deeper networks capture more complex relationships**:

- The first RNN layer might learn *local patterns* (like short-term dependencies).

- The next layer can combine them to learn *longer-term structures* or *semantic meaning*.
- The top layers learn *task-specific representations*.

This is like CNNs — lower layers detect edges, upper layers detect objects.

In RNNs — lower layers detect short patterns, upper layers detect meaning.

4. Notation: Understanding the Multi-layer Flow

At each time step t , for layer l :

$$h_t^{(l)} = f(W^{(l)}h_t^{(l-1)} + U^{(l)}h_{t-1}^{(l)})$$

Where:

- $h_t^{(l)}$ → hidden state at time t , layer l
- $h_t^{(l-1)}$ → output from previous layer
- $h_{t-1}^{(l)}$ → hidden state from previous time step
- $f()$ → activation (tanh, sigmoid, etc.)

So the data flows:

time dimension → across t

depth dimension → across layers (l)

This two-dimensional flow is what makes Deep RNNs powerful.

5. Why and When to Use Deep RNNs?

When to use:

- When a single RNN layer underfits (not learning enough patterns)
- When data has **hierarchical structure** (e.g. phonemes → words → sentences)
- When tasks require **deep temporal understanding**

Examples:

- **Speech recognition**
- **Machine translation**
- **Text summarization**
- **Video frame prediction**
- **Stock price or weather forecasting**

Be careful:

- Too many layers → harder to train (vanishing gradients)
- More parameters → slower training, risk of overfitting

6. Demo Code (Keras Example)

Here's how you create **Stacked LSTMs / GRUs** in Keras:

❖ Stacked LSTM:

```
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import LSTM, Dense

model = Sequential([
    LSTM(128, return_sequences=True, input_shape=(timesteps, features)),
    LSTM(64), # second LSTM layer
    Dense(10, activation='softmax')
])
model.summary()
```

❖ Stacked GRU:

```
from tensorflow.keras.layers import GRU

model = Sequential([
    GRU(128, return_sequences=True, input_shape=(timesteps, features)),
    GRU(64),
    Dense(10, activation='softmax')
])
model.summary()
```

`return_sequences=True` ensures each layer passes *all time steps* to the next layer. Without it, only the last time step is passed, and stacking won't work properly.

7. Variants

You can mix and match:

- **Stacked Simple RNNs**
- **Stacked LSTMs**
- **Stacked GRUs**
- **Hybrid models** (LSTM + GRU layers)

Also, **Bidirectional Stacked RNNs** exist, where each layer looks both forward and backward in the sequence.

8. Disadvantages

Challenge	Explanation
Vanishing/Exploding Gradients	More layers = harder backpropagation through time

High Computational Cost	Each layer adds parameters and training time
Overfitting	More layers may overfit on small datasets
Harder to Tune	More hyperparameters to manage (units, layers, dropout, etc.)

Solutions:

- Use **LSTM/GRU** (they already reduce gradient problems)
- Add **Dropout or Recurrent Dropout**
- Use **Layer Normalization**
- Limit layers to 2–3 for most tasks

9. Summary Table

Concept	Description
Deep RNN	Multiple RNN layers stacked vertically
Purpose	Capture hierarchical temporal features
Common Variants	Stacked LSTM, Stacked GRU
Key Benefit	Better feature abstraction
Main Risk	Vanishing gradients, overfitting
Implementation Tip	Use return_sequences=True for all layers except last

10. Intuitive Analogy

Think of Deep RNNs like a human language learner:

- **Layer 1:** Hears words (short-term memory)
- **Layer 2:** Understands grammar (medium-term patterns)
- **Layer 3:** Understands meaning or context (long-term patterns)

Each layer builds on the last to form a **deeper understanding** of the sequence.

11. In Summary

Deep or Stacked RNNs are powerful extensions of simple RNNs where multiple recurrent layers are stacked to learn complex temporal hierarchies.

They're especially useful for tasks involving multiple time scales but must be used carefully to avoid overfitting and vanishing gradients.